



Meowtion

On-device behaviour recognition for early feline health insight

Jerome Graves and Rose Delcour-Min

Revision 2.0 • June 2026

Abstract. Meowtion is a smart cat collar that infers a cat's health from changes in its daily habits. Cats instinctively hide illness, so the earliest warning is usually a quiet shift in routine, eating less, drinking more, or grooming and moving less, rather than an obvious symptom. A battery-powered collar classifies behaviour on the device itself with a confidence-gated cascade of two quantised neural networks, relays only the result over Bluetooth Low Energy to a mains-powered base station, and the station forwards it to a cloud backend that drives a live, owner-facing dashboard with longitudinal health analytics. The behaviour set is not hard-coded: owners define the actions to recognise, label captured clips, and a cloud trainer produces tiny models that are delivered back to the collar over the air. This document is the end-to-end technical reference for the system, covering the architecture and data flows, the hardware, the firmware on both microcontrollers, the on-device AI, the training pipeline, the cloud backend, the dashboard, the communication protocols, and the security and privacy model.

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Approach	3
1.3	Design principles	3
1.4	Contributions	4
1.5	Scope and structure of this document	4
2	System architecture and data flow	4
2.1	Topology	4
2.2	Component responsibilities	5
2.3	Operating modes	5
2.4	End-to-end data flows	5
2.5	Multi-tenant data model	7
2.6	Technology summary	7
3	Hardware	7
3.1	Collar	7
3.2	Station	7
3.3	Enclosure and assembly	8
3.4	Why split the system this way	9
4	Collar firmware	9
4.1	Responsibilities	9
4.2	Source layout	9
4.3	Streaming and threading	10
4.4	Model storage and flash layout	10
4.5	Build configuration	10
4.6	Development console and flashing	11
4.7	Current state	11
5	Station firmware	11
5.1	Source layout	11
5.2	Provisioning	11
5.3	Relay and clip upload	12
5.4	Over-the-air model delivery	12
5.5	Robustness: scanning after an update	12
5.6	Build and flash	13
6	On-device AI	13
6.1	Confidence-gated cascade	13
6.2	Matching training and inference	13
6.3	Runtime models and model-free safety	14
6.4	Memory budget	14
6.5	Operator set	14
7	Training pipeline	14
7.1	The loop	14
7.2	The trainer	15
7.3	Label-agnostic by design	15
7.4	Delivery contract	15

7.5	Known limitation	15
8	Cloud backend	16
8.1	Authentication	16
8.2	Realtime Database schema	16
8.3	Cloud Storage	16
8.4	Cloud Functions	16
8.5	Demo data simulator	17
8.6	Weather context	17
8.7	Configuration as a trust boundary	17
9	Dashboard	17
9.1	A hybrid front end	17
9.2	Login and session	18
9.3	Code structure	18
9.4	Live view versus history	18
9.5	Health analytics	18
9.6	Developer console	19
9.7	Read-only demo	19
10	Communication protocols	19
10.1	Telemetry advertisement	19
10.2	Audio clip-streaming frame	20
10.3	GATT services	20
10.4	Over-the-air model protocol	20
10.5	Cloud transport	21
11	Security and privacy	21
11.1	Audio never leaves the collar	21
11.2	Per-owner isolation	21
11.3	Devices hold tokens, not credentials	21
11.4	The demo is read-only by enforcement	22
11.5	What is and is not a secret	22
11.6	Owner data rights	22
12	Future work	22
13	References	23

1 Introduction

1.1 Motivation

Cats are evolved to conceal illness. A cat that feels unwell will mask it for as long as it can, so by the time a problem is obvious to an owner it is often already advanced. The early signal is rarely a dramatic symptom; it is a quiet shift in routine. A cat that begins to drink noticeably more, eats less, stops grooming, or sleeps through activity it used to enjoy is communicating a change. The difficulty is that no person watches their cat continuously, and day-to-day memory is a poor baseline. The change is real but invisible.

Meowtion makes that change visible. It observes the behaviours that make up a cat's routine, continuously and unobtrusively, and surfaces the trend to the owner. The goal is explicitly *not* to diagnose; it is to notice a routine change early enough that the owner can act on it, rather than only seeing the crisis. The clinical literature on feline medicine is consistent on the point that appetite, thirst, grooming and activity are among the first things to shift when a cat becomes unwell, and that these shifts are easy to miss without a continuous, objective baseline.

1.2 Approach

The system is a wearable plus a small fixed gateway. A lightweight, battery-powered collar carries the sensors and the intelligence: it classifies behaviour on the device itself, in real time, and never streams raw sensor data off the cat. It reports only a compact result (for example, “eating, high confidence”) over Bluetooth Low Energy (BLE) to a mains-powered base station in the home. The station bridges BLE to the internet over Wi-Fi and forwards results to a cloud backend, which drives a hosted dashboard where the owner sees their cat's activity and trends live, with the longer-run habit changes that matter for health flagged automatically.

1.3 Design principles

Four commitments shape the whole system and recur throughout this document.

Privacy by construction.

Classification happens on the collar. The microphone is used only to help recognise behaviour on-device; raw audio is processed and discarded, never recorded or transmitted in normal operation. Each owner's data is scoped to their own account by the backend's security rules (section 11).

Trainable, not fixed-function.

The recognised behaviours are defined by the owner, not baked into firmware. Owners capture and label example clips, a cloud trainer produces tiny models, and those models are delivered back to the collar over the air. Meowtion is a platform for recognising any pet behaviour, not a three-trick gadget (section 7).

Intelligence at the edge.

Inference runs on a microcontroller on the cat, not in the cloud. This keeps the radio uplink tiny (a classification result, not a sensor stream), keeps average power low enough for a collar-sized battery, and removes the cloud from the sensing loop entirely (section 6).

Health-first analytics.

The dashboard is built around longitudinal habit tracking and baseline comparison, not just a live read-out, so that a sustained change in a key habit is surfaced as an insight rather than buried in raw events (section 9).

1.4 Contributions

Considered as an engineering artefact, Meowtion’s notable elements are: a *confidence-gated, dual-model cascade* that runs an always-on motion classifier and invokes a short audio classifier only to resolve genuinely ambiguous behaviours (eating and drinking look alike to an accelerometer but not to a microphone), so audio is a deployed disambiguation signal rather than only a training aid; a *label-agnostic* pipeline in which the behaviour set is data-defined end to end, from the dashboard’s action list through the cloud trainer to the index-based on-collar classifier; an *over-the-air model path* that delivers quantised models to a Bluetooth-only wearable through the station; and a privacy and multi-tenancy model in which battery devices hold only a scoped, revocable token and never the owner’s credentials.

1.5 Scope and structure of this document

This document is the complete technical reference for Meowtion as built. It is organised so that a reader can either follow it top-to-bottom or jump to one tier. Section 2 gives the system-level picture, the component responsibilities, the two operating modes, and the end-to-end data flows, and is the right place to start. The detailed chapters then proceed bottom-up: the physical hardware (section 3); the firmware on each of the two microcontrollers (sections 4 and 5); the on-device AI (section 6); the training pipeline that produces and delivers models (section 7); the cloud backend (section 8); the dashboard (section 9); the communication protocols that tie the tiers together (section 10); and the security and privacy model (section 11). Section 12 records known limitations and planned work.

2 System architecture and data flow

This section is the map for the rest of the document. It introduces the four tiers and what each is responsible for, the two operating modes the system runs in, and then walks the four end-to-end data flows that together cover everything the system does: live monitoring, training capture, device onboarding, and model delivery.

2.1 Topology

Meowtion is a four-tier system, fig. 1: the **collar** on the cat, the **station** in the home, the **cloud** backend, and the owner’s **dashboard**. The defining constraint is that the collar is BLE-only and battery-powered: it has no route to the internet of its own. The mains-powered station exists to bridge that gap, giving the collar a cheap, always-on Bluetooth peer and a Wi-Fi path to the cloud. Everything above the collar is conventional web infrastructure; everything at the collar is shaped by power and radio budget.

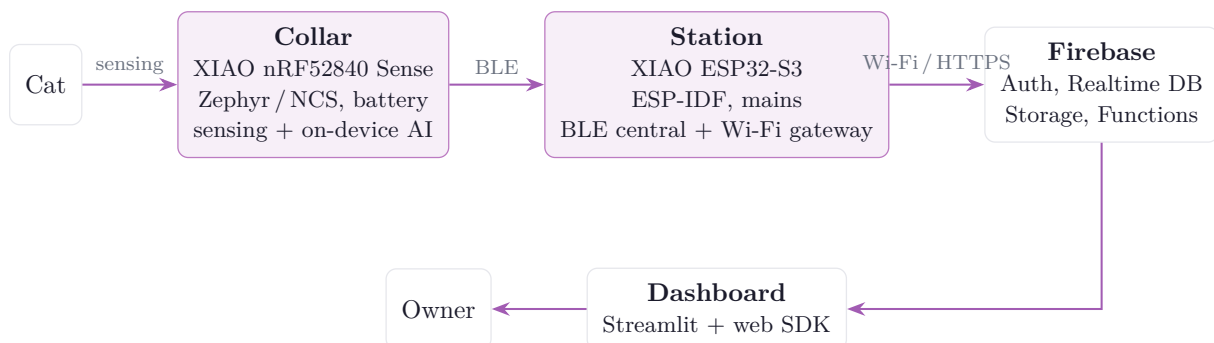


Figure 1: End-to-end topology. The collar is BLE-only, so the always-on station is its gateway to the cloud. The dashboard is a separate client of the same backend.

2.2 Component responsibilities

Table 1: What each tier is responsible for.

Tier	Responsibility
Collar	Sense (IMU + microphone), classify behaviour on-device, advertise the result as BLE telemetry, and, when a station subscribes, stream paired audio + IMU clips for training and accept models over the air. Holds no credentials.
Station	Scan for and relay registered collars to the cloud; capture and upload training clips; fetch trained models from Storage and push them to the collar over BLE. Authenticates to the cloud with a per-device token, not a login.
Cloud	Firebase: authenticate owners, store the live data and history (Realtime Database), store training clips and trained models (Storage), and run server-side logic (Cloud Functions for training and authenticated clip upload). Enforces per-owner isolation.
Dashboard	The owner-facing web app: a live current-state view and longitudinal health analytics, plus a static front door for sign-in and device registration and a dev console for data capture and training.

2.3 Operating modes

A station runs in one of two modes at a time, selected per station from the dashboard. The mode governs what crosses the BLE link and what the station does with it.

Production mode

The everyday monitoring mode. The collar classifies on-device and advertises only the result; the station reads that result and relays it to the cloud. No audio or raw motion ever leaves the collar. This is the privacy-preserving default.

Training mode

The data-collection mode. While a registered collar is in range, the station subscribes to the collar’s audio service; the collar streams paired audio + IMU clips, which the station slices, uploads, and the owner labels and trims in the dev console. This is how a new behaviour is taught. It is used deliberately and temporarily, not continuously.

2.4 End-to-end data flows

The system’s behaviour is fully described by four flows. Each is given below as the concrete sequence of steps across the tiers; the wire formats referenced are specified in section 10 and the security rules in section 11.

Flow 1 — Live monitoring (production). The steady state of a deployed system.

1. The collar samples its IMU continuously and runs the on-device cascade (section 6); when the motion result is ambiguous it briefly samples the microphone and runs the audio model to confirm.
2. The collar encodes the outcome (class index, confidence, step count, battery) into an 8-byte BLE telemetry advertisement (section 10). Its BLE address identifies which collar it is, so no identifier is carried in the payload.

3. The station, scanning continuously, hears the advertisement from a *registered* collar, timestamps it, maps the class index to the owner's label set, and writes the current state plus an event record to the Realtime Database over authenticated HTTPS.
4. The dashboard, reading the same database, refreshes the live card within seconds and folds the new event into the history and health analytics (section 9).

Flow 2 — Training capture. How labelled data is collected to teach a behaviour.

1. In the dev console the owner puts a station into training mode and selects the action being recorded (the “live label”).
2. The station connects to the in-range collar and subscribes to its audio characteristic. The collar streams continuously in 100ms frames, each frame a header followed by an audio payload and a time-aligned IMU payload (section 10).
3. The station concatenates frames, slices them into fixed-length clips, and POSTs each clip's bytes to the `upload_clip` Cloud Function with its device token. The function verifies the token's owner and writes the clip to that owner's Storage area; the station records the clip's metadata (label, paths, IMU presence) in the database.
4. The owner reviews clips in the dev console: trims the waveform, confirms or changes the label, and discards bad takes. The labelled clips are the training set for Flow 4.

Flow 3 — Device onboarding. How a station and collar are registered to an owner, performed once per device.

1. The owner signs in on the static front door and connects the station over USB using the browser's Web Serial API. The page mints a random per-device *token*, records it under the owner's account (`deviceTokens/<token>/owner`) and as a station device entry, and hands the station its Wi-Fi credentials, the owner id, and the token over the serial link. The station stores these in non-volatile storage; the token is its sole credential.
2. The station comes online, syncs time, and begins scanning. It publishes any unregistered collars it hears to its **seen** list.
3. The owner registers a heard collar with one tap in the dashboard. The collar then appears as a device on the account and is relayed by the station. Deleting the token revokes the device.

Flow 4 — Model training and over-the-air delivery. How labelled clips become a model running on the collar.

1. The owner triggers training from the dev console. The `train` Cloud Function reads the account's labelled clips, derives the class set from the labels present, trains an int8 IMU model and an int8 audio model, uploads them to Storage under `models/<uid>/`, and bumps the account's model version (section 7).
2. The station learns a new version is available (the version is mirrored into its own device config), downloads each model from Storage, and, over a brief dedicated BLE connection, streams it to the collar with a CRC-checked `begin/data/end` protocol (section 10).
3. The collar stages the model into its dedicated flash partition, verifies the CRC, loads it, and switches from advertising simulated state to advertising real classifier output. The whole path requires no firmware reflash.

2.5 Multi-tenant data model

Meowtion is multi-tenant. Each owner creates an account and registers their own station(s) and collar(s); all of an owner's data lives under `users/<uid>/` in the Realtime Database, and the database and storage rules scope every read and write to the owning account (sections 8 and 11). Isolation is enforced by the backend, not the client. Battery devices never hold a login: a station authenticates with its scoped, revocable token, and a collar holds nothing at all (the station relays it). A single public, read-only demo account, designated in server-side config, lets anyone explore a populated dashboard without signing up.

2.6 Technology summary

Table 2: Technology at each tier.

Tier	Stack
Collar	Seeed XIAO nRF52840 Sense; Zephyr / nRF Connect SDK (v3.3.1); TensorFlow Lite Micro with CMSIS-NN kernels
Station	Seeed XIAO ESP32-S3; ESP-IDF; NimBLE (BLE central) + Wi-Fi station
Cloud	Firebase: Authentication, Realtime Database, Cloud Storage, Python Cloud Functions (2nd gen)
Dashboard	Streamlit (Python) + Firebase web SDK (JavaScript); hosted on Streamlit Community Cloud
Training	TensorFlow / Keras, executed server-side in a Cloud Function; int8 post-training quantisation to TensorFlow Lite

3 Hardware

The build is two devices: the collar worn by the cat and the station plugged in at home. There is no custom PCB. Each device is a Seeed Studio XIAO module in a 3D-printed shell, chosen for the small footprint, integrated radios, hand-solderable castellated pads, and, on the collar, on-module sensors. The complete parts list, print files and assembly steps live in `hardware/README.md`; this section summarises the design and the reasoning behind it.

3.1 Collar

The collar is built around the Seeed XIAO nRF52840 Sense. The nRF52840 provides a Bluetooth Low Energy radio and enough flash and RAM to run quantised neural-network inference on its Cortex-M4F. Critically for Meowtion, the *Sense* variant carries both sensors the system needs on the module itself, a six-axis IMU (LSM6DS3TR-C) and a PDM MEMS microphone, so the collar needs no external sensor breakout and stays small and light.

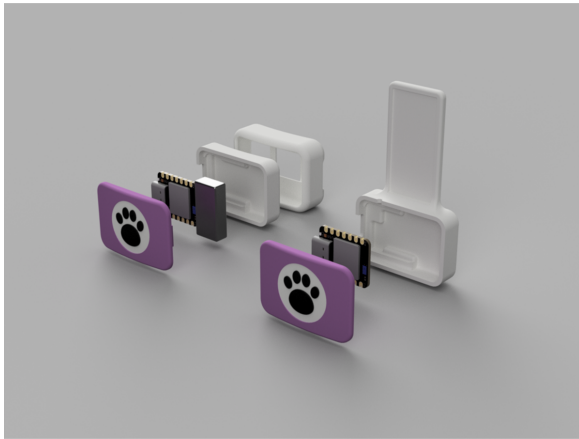
The IMU supplies the motion stream that is always classified; the microphone supplies the short audio window used to disambiguate behaviours that look alike by motion but differ by sound (section 6). The microphone port sits behind the PTFE membrane in the front shell; correct membrane placement matters for audio quality and water resistance and is documented in the hardware guide. The collar runs on its own 1S cell, charged through the XIAO's onboard divider.

3.2 Station

The station is a Seeed XIAO ESP32-S3. The ESP32-S3 has both Wi-Fi and BLE, so a single module acts as the BLE central that listens to the collar and the Wi-Fi client that reaches the

Table 3: Collar bill of materials (one per cat).

Part	Qty	Role
Seeed XIAO nRF52840 Sense	1	SoC (BLE), on-module IMU and PDM microphone; runs inference
1S LiPo cell, 3.7 V 100 mAh, with protection (BMS)	1	Powers the wearable; soldered to the BAT pads. A cell with a built-in protection circuit is required
20 mm hydrophobic PTFE filter membrane	1	Sits over the mic hole: passes sound, keeps water and dust out
10 mm quick-release safety collar	1	Standard strap; the silicone skin mounts to it
3D-printed shells + TPU skin	1	PETG front and back shells; a flexible TPU skin holds the sealed assembly on the strap



(a) Exploded view: boards drop into the back shells, front covers clip on.



(b) Assembled internals.

Figure 2: Collar and station hardware. Full parts, print files, and assembly steps are in `hardware/README.md`.

cloud. It is mains-powered over USB-C, so it can scan and stay online continuously without the power constraints of the wearable. The station carries no sensors of its own; its job is to hear the collar, capture training clips when asked, relay results to the cloud, and push models to the collar.

3.3 Enclosure and assembly

The front shell is shared between the collar and the station; the back shells and the collar’s TPU skin are per-device. The collar shells are PETG for rigidity and the skin is flexible TPU; the station back shell is any rigid filament. The collar is assembled by seating the board and cell in the back shell, fitting the PTFE membrane over the mic hole, bonding the shell halves with water-resistant glue, and slipping the sealed unit into the TPU skin on a quick-release strap. The station is simply seated, glued closed, and plugged in by the bowl. The hardware designs are released under CERN-OHL-S v2 (strongly reciprocal open hardware).

3.4 Why split the system this way

The collar must be small, light, and run for a useful time on a small cell, which rules out Wi-Fi on the wearable: its power draw is incompatible with a collar-sized battery. BLE is low-power but short-range and is not an internet transport. The fixed, mains-powered station resolves both problems: it gives the collar a cheap always-on BLE peer and a Wi-Fi path to the cloud. The split also keeps the privacy boundary clean, as the heavy and sensitive processing (including all audio) stays on the collar and only a classification result crosses to the station.

4 Collar firmware

The collar runs on Zephyr [1] via the nRF Connect SDK (NCS v3.3.1), targeting the board `xiao_ble/nrf52840/sense`. It is a connectable BLE peripheral that senses on the cat, advertises a compact telemetry packet, and, when a station connects and subscribes, streams paired audio + IMU clips for training. It never touches Wi-Fi or the internet, which keeps it small and low-power and lets it always run on its own battery. The build is driven by `build.ps1`, which locates the local NCS toolchain, runs `west build -b xiao_ble`, and copies out the UF2.

4.1 Responsibilities

The firmware has four jobs, driven from the main loop and BLE callbacks:

1. Sample the IMU continuously (and the microphone when needed) and run the classifier cascade.
2. Advertise the result as an 8-byte BLE telemetry packet for the station to read (section 10).
3. In training mode, stream paired audio + IMU frames to a subscribed station for capture.
4. Receive new models over the air and swap them in safely.

4.2 Source layout

Table 4: Collar firmware modules (`firmware/collar/src/`).

File	Responsibility
<code>main.c</code>	Boot and the telemetry loop; orchestration only
<code>ble.c</code>	Advertising, identity, the audio GATT service, notifications
<code>audio.c</code>	PDM microphone capture, decimated to 8 kHz μ -law
<code>audio_codec.h</code>	μ -law codec and the model-input audio representation
<code>streaming.c</code>	Decoupled reader/sender threads that assemble and send clip frames
<code>imu.c</code>	LSM6DS3TR-C continuous sampler
<code>battery.c</code>	1S LiPo charge state via the onboard divider
<code>classifier.c</code>	The confidence-gated cascade policy (IMU first, audio confirms)
<code>tflm_classifier.cpp</code>	TensorFlow Lite Micro backend for the cascade
<code>model_loader.h</code>	Runtime model-install API (<code>clf_set_imu_model</code> / <code>clf_set_audio_model</code>)
<code>production.c</code>	Rolling IMU window; runs inference and writes real telemetry
<code>ota.c</code>	OTA receive state machine; stages models to flash and defers the swap

The classifier is split deliberately. `classifier.c` defines the cascade policy in plain C with *weak* hooks for model-presence queries and inference calls; `tflm_classifier.cpp` provides the *strong* `extern "C"` definitions that override them when the TensorFlow Lite Micro backend is

compiled in. This keeps the policy readable and separable from the heavy C++ inference, and it means the firmware links and runs even with the backend or any model absent.

4.3 Streaming and threading

Capturing training data is the collar’s most timing-sensitive path: PDM audio arrives in a steady stream and must be paired with IMU samples and pushed over BLE without dropping frames. The reader and sender are therefore decoupled (`streaming.c`): one side assembles 100 ms frames from the audio and IMU sources, the other side sends them over the audio characteristic, so a momentarily slow radio cannot stall capture. The collar negotiates a large ATT MTU (247 bytes) and uses the 2 Mbit PHY to keep throughput up.

4.4 Model storage and flash layout

Models are not linked into the firmware image. They live in a dedicated flash partition so a new model can be written over the air without reflashing. The partition map is fixed in `pm_static.yml` to stop the partition manager from silently regrowing the application region over the model storage:

Table 5: Collar flash partitions (`pm_static.yml`).

Partition	Address / size	Contents
app	0x27000 / 0x85000	Application code (fixed size)
models_storage	0xAC000 / 0x40000 (256 KiB)	Header plus the IMU (slot 0) and audio (slot 1) model blobs
settings_storage	0xEC000 / 0x08000	NVS settings
uf2_partition	0xF4000 / 0x0C000	UF2 bootloader (unchanged)

4.5 Build configuration

Enabling on-device inference meant pulling TensorFlow Lite Micro into the NCS workspace (an optional, normally-excluded west module) and turning on C++17 and the TFLM and CMSIS-NN options. Two stability fixes are baked into the configuration: TFLM’s `AllocateTensors()` and `Invoke()` overflow Zephyr’s small default stack, so the main stack is raised; and the model storage uses `stream_flash` to lazily erase pages during an OTA write.

```

1 CONFIG_CPP=y
2 CONFIG_STD_CPP17=y
3 CONFIG_TENSORFLOW_LITE_MICRO=y
4 CONFIG_TENSORFLOW_LITE_MICRO_CMSIS_NN_KERNELS=y
5 CONFIG_MAIN_STACK_SIZE=16384      /* AllocateTensors()/Invoke() overflow the ~2 KiB
   default */
6 CONFIG_STREAM_FLASH_ERASE=y        /* lazily erase model pages during OTA */
7 CONFIG_BT_L2CAP_TX_MTU=247        /* large MTU for audio streaming + OTA chunks */
8 CONFIG_BT_CTLR_PHY_2M=y           /* 2 Mbit PHY for throughput */

```

Listing 1: Inference-related `prj.conf` settings (collar).

Because committing a model touches a lot of stack, the OTA receive path does not load the model from inside the BLE-RX callback. It stages the bytes to flash, then defers the verify-and-swap onto the main thread (section 10), so a model can be installed without starving the BLE stack.

4.6 Development console and flashing

The XIAO nRF52840 has no debug COM port, so flashing is always a UF2 drive-copy: double-tap RESET to mount the bootloader drive, then re-run the build script to copy the UF2 across. For development the firmware brings up a USB CDC ACM console on the *legacy* USB device stack (the newer NEXT stack's CDC console silently drops output on this board). On that console the collar prints a `MEOW> collar id=...` banner and a periodic `state= steps= batt=` line; the `id=` banner is what the dashboard reads over Web Serial to register the collar.

4.7 Current state

Telemetry state, activity and step count are currently produced by a dwell-based behaviour state machine (real battery is wired) and switch to live classifier output once a model is loaded; until then the cascade reports UNKNOWN and the firmware runs model-free and safe (section 6). The cascade itself already runs at capture time, on the IMU window paired with each clip, so the inference path is exercised before any model ships.

5 Station firmware

The station runs on ESP-IDF [2] on the XIAO ESP32-S3. It is a BLE central that hears registered collars and a Wi-Fi gateway that relays them to the owner's Firebase account. It also captures short audio+IMU clips for training and uploads them, and it delivers trained models to the collar over BLE. It has no Firebase login: Wi-Fi credentials, the owner id, and a per-device token are provisioned over USB serial and stored in non-volatile storage (NVS), and the database authorises the station because that token is registered to the owner.

5.1 Source layout

Table 6: Station firmware modules (`firmware/station/main/`).

File	Responsibility
<code>main.c</code>	Boot and the roughly one-second orchestration loop
<code>config.c</code>	Provisioned credentials, NVS storage, USB-serial provisioning
<code>board.c</code>	Power-source detection (USB vs battery)
<code>wifi.c</code>	Wi-Fi station bring-up and time synchronisation
<code>cloud.c</code>	The Firebase Realtime Database / HTTP layer (the only HTTP in the firmware)
<code>weather.c</code>	Local weather from the Open-Meteo feed
<code>ble.c</code>	The whole NimBLE collar subsystem (scan, relay, audio capture) behind <code>ble_service()</code>
<code>ota.c</code>	OTA model delivery: pushes trained <code>.tflite</code> models to the collar over BLE

5.2 Provisioning

Provisioning is a single JSON line sent over USB serial; the dashboard's "Connect a device" form does this automatically over Web Serial, but it is plain text and can be sent by hand for debugging:

```
1 {"ssid":"..","pass":"..","owner":"<uid>","token":"<random>","name":"..","lat":50.8,"lon":-0.1}
```

Listing 2: Provisioning line (USB serial).

The station stores these in NVS and can be re-provisioned at any time from the dashboard, or wiped with `idf.py erase-flash`. The token is the station's only credential; deleting it from the owner's account revokes the device immediately. Repository secrets such as `secrets.h` are excluded by `.gitignore`, so no live credential is tracked.

5.3 Relay and clip upload

In production mode the station scans continuously, hears the telemetry advertisements of registered collars, maps the compact result (mode, class index, confidence) onto the cat's record, and writes both a live `current` state and discrete activity events as episodes begin and end (`cloud.c`). Events carry the model version and class index rather than a resolved name, so the dashboard can map them to labels even as the label set changes (section 8). In training mode it connects to an in-range collar, captures clips, and POSTs each clip's bytes to the `upload_clip` Cloud Function with its device token; the function verifies the token's owner and writes the file to that owner's Storage area, since clients cannot write Storage directly. A manual capture path ignores the signal-strength gate, used to collect behaviours such as purring that happen while the cat rests rather than at the bowl.

5.4 Over-the-air model delivery

The station is the delivery vehicle for trained models, since the collar is BLE-only and never touches Storage. The wire protocol is fixed in the shared header `firmware/common/meow_ota.h`: the collar is the GATT server that stages a `.tflite` into flash, the station is the client that pushes it (section 10).

Version gate.

The `train` function bumps `users/<uid>/models/version` and mirrors it into this station's own config as `modelVer` (the station may read only its own `users/<uid>/devices/<token>/subtree`, not `users/<uid>/models`). The station keeps its last-pushed version in NVS; a push is due when the config version is higher.

Download.

For each slot it fetches the model from Storage at the public-read path `models/<uid>/imu_model.tflite` (IMU slot) or `models/<uid>/audio_model.tflite` (audio slot). A 404 means that slot has no model yet and is skipped.

Push.

It computes a CRC-32/IEEE over the bytes, then over a brief dedicated BLE connection writes `BEGIN`, streams the model to the data characteristic in write-with-response chunks (the chunk write-response is the flash-write flow control), and writes `END`. On the collar's `OK` it records the new version in NVS; on any error it leaves NVS unchanged and retries later.

A push runs only when a registered collar is in range and audio capture is idle, so model delivery never contends with the capture or relay path for the radio.

5.5 Robustness: scanning after an update

The station drives one BLE radio for several jobs, and the OTA path exposed a subtle defect. To push a model the station must cancel scanning and open a connection; the OTA path used its own GAP callback and originally never restarted scanning when it finished, so after an update the station went deaf and the dashboard reported the collar offline. The fix is an idempotent `ble_resume_scan()`, guarded on the discovery-active flag, that the OTA path calls on both completion and bail-out. It was verified on hardware.

5.6 Build and flash

The station builds with `idf.py build` and flashes over its USB serial port with `idf.py -p <COM> flash monitor` (the repository's VS Code task **Build+Flash Station** wraps this). It is verified building under ESP-IDF.

6 On-device AI

All classification happens on the collar, in real time, using TensorFlow Lite Micro [3] with the CMSIS-NN kernels [4] for the nRF52840's Cortex-M4F. The models are tiny int8 networks in the spirit of the TinyML approach [5]: small enough to fit a microcontroller's RAM and run within the collar's power budget, yet accurate enough to separate everyday behaviours.

6.1 Confidence-gated cascade

Two behaviours can look almost identical to a motion sensor while sounding completely different: eating and drinking both put the head down at a bowl with similar gross motion. Rather than force a single model to separate them from accelerometry alone, Meowtion runs a confidence-gated cascade of two models:

1. The **IMU model** runs continuously on the motion stream. It is cheap and always on, and it carries most of the classification load.
2. When the IMU model is *unsure* (its top class falls below a confidence threshold), the short **audio model** is invoked to confirm. Audio is therefore a deployed disambiguation signal, not merely a training aid.

This keeps average power low, since the audio path runs only when it is needed to break a tie, while recovering accuracy that motion alone cannot provide. The cascade policy lives in plain C (`classifier.c`) and is independent of the inference backend, so the gating logic can be read and tuned without touching the TensorFlow Lite Micro code.

6.2 Matching training and inference

The single most important correctness property is that the on-device input transform must mirror the transform used during training, exactly. A model trained on one representation and fed another at inference produces confident nonsense. Meowtion enforces the match on both paths:

- **Motion.** The rolling window is normalised per channel (subtract each channel's mean over the window, divide by the global maximum absolute value with a small epsilon) and quantised to int8 using the input tensor's own scale and zero-point.
- **Audio.** Inference audio is decimated and encoded to match the training representation: 8 kHz μ -law, produced by the same conversion used to prepare the training clips (section 10). A mismatch here was an early source of confident misclassification.

To avoid a large floating-point scratch buffer on a RAM-constrained part, the IMU path quantises directly from the int16 source in a few passes (per-channel mean, then global max-abs, then write int8) rather than materialising a float window, listing 3.

```
1 // pass 1: per-channel mean over the window
2 // pass 2: global max-abs of (sample - channel_mean)
3 // pass 3: int8 = round( x_norm / scale ) + zero_point,
4 //           where x_norm = (sample - channel_mean) / (maxabs + 1e-6)
```



```
5 // scale and zero_point are read from the input tensor itself.
```

Listing 3: Direct int16-to-int8 quantisation, mirroring training (sketch).

6.3 Runtime models and model-free safety

No model is compiled into the firmware. Each stage loads its `.tflite` at runtime (`clf_set_imu_model` / `clf_set_audio_model`), from the dedicated flash partition written over the air (sections 4 and 10). Until a model lands the engine runs model-free: the presence queries return false and the cascade returns `UNKNOWN`, so the firmware is safe to run with no model at all and simply advertises a simulated state until a real model is installed.

6.4 Memory budget

TensorFlow Lite Micro uses a fixed, statically allocated tensor arena per model (this firmware uses no heap). The arenas dominate the RAM cost of inference and are sized to the two networks. The values in table 7 are the starting budgets; each must be tuned to the real exported model, since an undersized arena makes `AllocateTensors()` fail and the stage report itself absent.

Table 7: Tensor arenas (initial budgets).

Model	Arena	Notes
IMU (slot 0)	16 KiB	Always-on motion classifier, a small temporal conv net
Audio (slot 1)	48 KiB	Larger 1-D conv over 8000 points; invoked only to confirm

Enabling inference initially pushed RAM use to roughly 94%; eliminating the float-window scratch buffer (quantising directly from int16) brought it back into the low 80s, leaving headroom for the BLE stack and the raised main stack (section 4).

6.5 Operator set

The op resolver is a fixed `MicroMutableOpResolver`, so only the operators the models actually use are linked into the image. The Keras `Conv1D` layers export as `CONV_2D` wrapped in `EXPAND_DIMS` and `SQUEEZE`; the missing `EXPAND_DIMS` operator was the cause of an early “didn’t find op” load failure, and the resolver now includes it alongside the pooling, fully-connected, reshape, softmax, and quantise/dequantise operators the networks need. Pinning the operator set this way keeps the binary small and makes a load failure a clear, diagnosable event rather than a silent fallback.

7 Training pipeline

Meowtion learns new behaviours through a closed loop that an owner drives entirely from the dashboard, with no local toolchain: capture, label, train, deliver, run. The loop is what turns the collar from a fixed-function sensor into a platform for recognising any behaviour the owner can demonstrate.

7.1 The loop

1. **Define behaviours.** The owner lists the actions to recognise (eat, drink, purr, scratch, litter-tray, and so on) in the dev console. This set drives both the clip-label dropdown and the trained model’s outputs.

2. **Capture.** With a station in training mode, paired audio and time-aligned motion are recorded while a registered collar is in range and uploaded to Cloud Storage (Flow 2, section 2).
3. **Label.** The owner reviews clips in the dev console, sets each clip's action, trims dead space from the waveform, and discards poor examples.
4. **Train.** The `train` Cloud Function trains the models server-side from the labelled clips and writes the results back: the quantised `.tflite` blobs to Storage, and the new version and label list to the database.
5. **Deliver.** Each station watches a model-version key; when it changes the station downloads the new `.tflite` files and pushes them to the collar over the air (section 10).
6. **Run.** In production mode the collar classifies with the new model and the dashboard shows live, named behaviours.

7.2 The trainer

Training runs in the `train` Python Cloud Function, gated by a developer account (section 8). It reads the account's labelled clips (metadata from the database, audio and motion bytes from Storage), and derives the class set directly from the labels present, so the network's outputs follow whatever behaviours the owner defined. It then trains the two cascade stages independently:

- a small 1-D convolutional network over the motion windows (six IMU channels), and
- a larger 1-D convolutional network over the 8 kHz audio windows.

Each input is windowed with overlap and normalised exactly as the collar normalises it at inference, so the on-device transform and the training transform match (section 6). The trained Keras models are converted to TensorFlow Lite with full int8 post-training quantisation using a representative dataset, so the exported `.tflite` weights and activations are int8 and the input and output tensors carry the scale and zero-point the collar uses to quantise its own inputs.

7.3 Label-agnostic by design

The trainer is not hard-coded to a fixed set of actions; it learns whatever label set the owner defines. This is what makes Meowtion a platform rather than a fixed-function gadget: the same pipeline that recognises eating and drinking can be pointed at any distinguishable behaviour the owner can capture and label. The class set flows end to end as data, from the dashboard's action list, through the trainer, to the index-based on-collar classifier, with names attached only at the edges.

7.4 Delivery contract

Training outputs are written to predictable locations so the station finds them without coordination: the model blobs land in Storage as `models/<uid>/imu_model.tflite` and `models/<uid>/audio_model.tflite`, the new version is mirrored into each of the owner's stations' configuration so their version gate fires, and the label list is stored in `users/<uid>/models` so the dashboard can map a class index back to a name across label-set changes (section 8).

7.5 Known limitation

A model trained on only two classes (for example, eat versus drink) has no way to represent "neither" and will confidently assign one of its two labels to an idle cat. The fix is to retrain with an explicit idle/other class; this is a data and modelling task, not a firmware change, and is tracked in section 12.

8 Cloud backend

The backend is Firebase [6]. It provides owner authentication, a Realtime Database for live state and history, Cloud Storage for training clips and model blobs, and Python Cloud Functions (2nd generation) for server-side logic. There is no custom server to operate, and the whole backend is self-contained in `app/firebase/`, deployed with `firebase deploy` from that folder.

8.1 Authentication

Owners sign in with Firebase Authentication (email/password or Google). The collar and station do *not* authenticate as users: a station carries a scoped per-device token, and a collar carries nothing (the station relays it). This keeps the credential surface on battery devices to a single revocable token (section 11).

8.2 Realtime Database schema

State and history live in the Realtime Database, keyed by owner so the security rules can scope every read and write to the owning account. Table 8 lists the main paths.

Table 8: Realtime Database paths.

Path	Contents
<code>users/<uid>/devices/<token></code>	A registered station (<code>type:station</code>) with its relayed <code>cats/<catId></code> subtree, or a registered collar entry
<code>.../cats/<catId>/current</code>	The cat’s live state (version, class, confidence, steps, battery)
<code>.../cats/<catId>/events</code>	Discrete activity events (carry model version + class index)
<code>.../devices/<station>/weather</code>	Current and recent local weather for the station’s location
<code>users/<uid>/models</code>	Current model version, status, and the label list
<code>users/<uid>/actions</code>	The behaviours to recognise (managed from the dev console)
<code>deviceTokens/<token>/owner</code>	Maps a device token to its owning account
<code>config/demoOwner</code>	UID whose data backs the public read-only demo
<code>config/devAccounts/<uid></code>	Accounts allowed to use the dev/training tools

Events store the model version and class index rather than a pre-resolved name. The dashboard maps the index to a label using that version’s label list, so historical events stay correct even as the owner changes their behaviour set.

8.3 Cloud Storage

Storage holds the training clips (audio plus aligned motion) and the trained model blobs, in an owner-scoped layout: clips at `training/<owner>/<collar>/<ts>.{wav,imu}` and models at `models/<uid>/{imu,audio}_model.tflite`. Training clips are private to the owner and are deliberately *not* world-readable, even in the demo, because home audio must never be public. Model blobs are public-read so the token-only station can fetch one to push to the collar, and are written only by the trainer.

8.4 Cloud Functions

Two functions provide the only server-side logic. Both run on the function’s ambient service account, so no service-account key is checked into the repository.

train

Server-side model training, POST with a developer account's Firebase ID token and gated additionally by `config/devAccounts/<uid>`. It reads the account's labelled clips, trains the int8 IMU and audio models (section 7), uploads them to `models/<uid>/`, updates `users/<uid>/models`, and mirrors the new version into each of the owner's station configs to trigger delivery.

upload_clip

Authenticated clip upload for the token-only station. The station has no login, so it cannot satisfy the Storage rules directly; it POSTs the clip bytes here with its device token, the function verifies `deviceTokens/<token>/owner`, and writes the file as administrator into that owner's Storage area. This lets the Storage rules deny all direct client writes while still letting an authorised station upload.

8.5 Demo data simulator

So that the public demo always presents a populated, believable dashboard, a scheduled Cloud Function maintains a fully simulated companion collar ("Purrminator") on the demo account. It is a standalone device that needs no station and is always online. On first run it backfills roughly six months of realistic activity history, weighted by time of day so the pattern matches an average cat (crepuscular, busy around dawn and dusk, resting overnight), with matching daily weather from a free historical feed. Thereafter it wakes on a short schedule and commits each activity as an event at the moment that action ends, so new events arrive at naturally varying intervals. The simulator only ever writes the demo account's own data and never touches a real collar.

8.6 Weather context

Local weather is incorporated so the dashboard can read a habit change against its environment, since a warm spell that increases drinking and suppresses appetite should not be mistaken for illness. The station polls a keyless forecast feed (Open-Meteo) for its provisioned location and writes current and recent weather under its device record; the demo simulator uses the same provider's historical archive to backfill weather alongside the simulated activity.

8.7 Configuration as a trust boundary

The `config` subtree is readable by clients but not writable by them; it is maintained from the Firebase console. This lets the system publish facts the clients need (which account backs the demo, which accounts are developers) without letting a client grant itself privileges.

9 Dashboard

The owner-facing application is a Streamlit [7] dashboard backed by a small set of static pages for the parts that need the Firebase web SDK directly (sign-in, account and device management, the developer/training console, and the privacy policy). It is hosted on Streamlit Community Cloud and deploys automatically from the repository's main branch.

9.1 A hybrid front end

Device registration needs the browser's Web Serial API, which cannot run in Streamlit's Python or its sandboxed component iframes, and authentication is easiest with the Firebase web SDK in the browser. The data views and charts, by contrast, are easiest in Streamlit's Python. Meowtion therefore serves both halves from the same origin so they share a login: the static

sign-in page writes the Firebase ID token into a same-origin cookie, and the Python dashboard reads that cookie to identify the owner.

Table 9: Dashboard surfaces.

Surface	Tech	Role
<code>app.py</code>	Streamlit/Python	Live cat state, activity history, health analytics
<code>account.html</code>	JavaScript	Sign-in, account management, device pairing over Web Serial
<code>dev.html</code>	JavaScript	Developer/training console: capture, label, train, mode
<code>privacy.html</code>	JavaScript	Privacy policy

9.2 Login and session

The owner logs in once on the front door with Firebase Auth; it writes the ID token into a same-origin cookie (`mtoken`). `app.py` reads that cookie client-side with the `extra-streamlit-components` cookie manager rather than the server-side cookie API, because Streamlit Community Cloud does not expose browser cookies to the server. Logged-out visitors get a sign-in gate and never see the dashboard; logging out navigates to the front door, signs out of Firebase, and clears the cookie. No token is ever placed in a URL. A separate flag cookie marks the public read-only demo.

9.3 Code structure

`app.py` is a thin entry point that only wires the pieces in order (theme, sign-in, live cards, analytics). Everything else is split into a `meowtion_dash` package so the analytics view can be edited without touching login or data-formatting code: `theme` (page config and brand header), `auth` (cookie/JWT to `(uid, token, is_demo)`), `firebase` (cached database reads), `data` (the clean activity DataFrame and filter helpers), `charts` (branded Altair charts), `live` (the live current-state cards), and `weather`. The editable analytics live in `dashboard_view.py`, which is handed a ready, one-row-per-event pandas DataFrame and never sees a cookie or raw Firebase JSON.

9.4 Live view versus history

The two reads have very different needs, and are split accordingly. The live current-state card refreshes every few seconds but needs only each cat’s `current` record and its most recent events, so it issues a small per-cat read. The full activity history (potentially months of events) is fetched on a longer cache interval and reused across refreshes. Separating the two keeps the fast-refreshing live card from re-downloading the whole history on every tick, which keeps the page light and the backend bandwidth low.

9.5 Health analytics

The analytics view is built around health, not just a live read-out. A *health watch* compares each key habit (eating, drinking, grooming, resting) over recent days against the cat’s own longer-run baseline, and flags a sustained change with a plain-language, vet-oriented note rather than a diagnosis. Two design choices keep it honest: it compares whole days only, since events are logged when an action ends and the current day is always partial; and it pairs a flagged change with local weather context, so a warm-spell rise in drinking is presented with its likely cause rather than as an alarm. A drill-down lets the viewer choose whether the x-axis spans a day, a

week, or a month and then which specific window to show, and the activity charts are stacked by behaviour and rendered on a transparent background so they sit on the page’s brand canvas.

9.6 Developer console

`dev.html` is visible only to developer accounts and is the data-collection and training control panel: live per-station capture status (signal, recording state, collar battery); the Training/Production mode switch; the editable list of actions to recognise; the labelled-clip review with waveform trimming; dataset statistics and a motion-variety measure to gauge whether a class has enough varied examples; and the button that triggers cloud training. It also offers a full-screen, phone-style capture view with large tappable action buttons for live labelling at the bowl.

9.7 Read-only demo

A public demo lets anyone explore a populated product without an account. It reads the designated demo owner’s data, which the security rules make world-readable, but every write control and function trigger is disabled and the dev console is available only in a read-only variant. The demo is genuinely read-only because it is enforced by the backend rules, not merely hidden in the interface (section 11).

10 Communication protocols

Four contracts tie the tiers together: the BLE telemetry the collar advertises, the audio clip-frame format it streams during training, the over-the-air model protocol, and the cloud HTTP it is all relayed over. The two BLE wire formats are defined once in shared headers (`firmware/common/meow_protocol.h` and `meow_ota.h`) that *both* firmwares include, so the collar (Zephyr) and station (ESP-IDF) builds cannot drift out of sync.

10.1 Telemetry advertisement

In production mode the collar publishes its result in the manufacturer-specific field of its BLE advertisement, so the station can read it passively without connecting. The packet is eight bytes (table 10); the collar’s BLE address identifies which collar it is, so no identifier is carried in the payload. The `version` byte selects how the two result bytes are read: version 1 is the simulated behaviour state machine, version 2 is real on-device classification.

Table 10: Eight-byte telemetry packet (manufacturer AD data).

Bytes	Field	Meaning
0–1	company id	0xFFFF (test/development identifier)
2	version	1 = simulated state, 2 = real on-device classification
3	state / class	v1: state (0 sleep ...5 groom); v2: predicted class index (0xFF = unknown)
4	activity / confidence	v1: activity level 0–100; v2: confidence 0–100
5–6	steps	step count, <code>uint16</code> , little-endian
7	battery	battery level 0–100

Because the class is an index, not a name, the station and dashboard resolve it through the label list for the running model version, which keeps history correct across label-set changes (section 8).

10.2 Audio clip-streaming frame

While a station is subscribed in training mode, the collar streams continuously on the audio characteristic in 100 ms frames. Each frame is a packed header followed by an audio payload and a time-aligned IMU payload; the station concatenates frames and slices them into fixed-length training clips.

```

1  #define MEOW_CLIP_MAGIC 0x574F454Du    /* 'MEOW' little-endian, marks each frame
    start */
2
3  struct __attribute__((packed)) meow_clip_hdr {
4      uint32_t magic;
5      uint8_t  version;    /* 1 = 16-bit PCM audio, 2 = 8-bit G.711 mu-law */
6      uint8_t  imu_axes;   /* 6 = accel x/y/z + gyro x/y/z */
7      uint16_t imu_rate_hz;
8      uint32_t audio_bytes; /* audio payload size (one 100 ms frame) */
9      uint32_t imu_bytes;  /* IMU payload size */
10     uint32_t audio_rate;  /* output sample rate, for the station's WAV header */
11 };
12 /* frame layout: [ meow_clip_hdr ][ audio payload ][ IMU payload ] */

```

Listing 4: Clip-frame header (firmware/common/meow_protocol.h).

To halve the data on the BLE link the collar encodes audio as 8-bit G.711 μ -law (version 2); the station decodes it back to 16-bit PCM for a normal WAV. The μ -law codec is defined inline in the same shared header, so the encode on the collar and the decode on the station are guaranteed to match, and this is the same representation the model is trained and run on (section 6).

10.3 GATT services

The collar exposes two custom 128-bit GATT services that share a common Meowtion base UUID: the *audio* service used for clip streaming, and the *OTA* service used for model delivery. The OTA service uses three UUIDs distinguished by a 0x0b** group, one each for its control, data, and status characteristics. Both firmwares derive these UUIDs from the same header, the collar through Zephyr’s encoding macro and the station from the raw little-endian byte arrays the header also provides.

10.4 Over-the-air model protocol

Model delivery is a small framed protocol over the OTA service. The client (station) writes opcodes to the *control* characteristic, streams the model to the *data* characteristic, and the collar acknowledges on a *status* notification. A transfer is: BEGIN (carrying the target slot, the total length, and a CRC), a stream of write-with-response DATA chunks (each chunk’s write response is the flash-write flow control), then END. The collar erases the slot on BEGIN, writes chunks as they arrive, and on END re-reads the model from flash, verifies the CRC against the announced value, and reports the outcome.

```

1  /* CONTROL characteristic: first byte is an opcode; BEGIN is followed by
    meow_ota_begin. */
2  enum { MEOW_OTA_OP_BEGIN = 1, MEOW_OTA_OP_END = 2, MEOW_OTA_OP_ABORT = 3 };
3
4  struct __attribute__((packed)) meow_ota_begin {
5      uint8_t slot;    /* 0 = IMU model, 1 = audio model */
6      uint32_t total_len; /* model size in bytes */
7      uint32_t crc32;   /* CRC-32/IEEE over the model blob */
8  };

```

```
9
10 /* STATUS characteristic (notify): the collar's acknowledgement. */
11 enum { MEOW_OTA_ST_READY = 1, MEOW_OTA_ST_RECEIVING = 2, MEOW_OTA_ST_OK = 3,
12        MEOW_OTA_ST_ERR_SIZE = 5, MEOW_OTA_ST_ERR_CRC = 7, MEOW_OTA_ST_ERR_SEQ = 8
        };
```

Listing 5: OTA control opcodes, BEGIN payload, and status codes (firmware/common/meow_ota.h).

Crucially, END verifies and stages the model but does *not* swap the live model from inside the BLE callback. Committing a model is stack-heavy, and doing it in the radio callback crash-looped the device; the actual swap is deferred to a service call on the main thread's larger stack (section 4). That split, together with the raised main stack, is what made OTA reliable on hardware. A persisted slot magic marks a slot that holds a valid committed model, so the collar can tell an empty slot from a written one across a reboot.

10.5 Cloud transport

Above BLE, the station speaks ordinary HTTPS to Firebase: it writes telemetry to the Realtime Database through its REST interface, authorised by its device token, and POSTs training-clip bytes to the `upload_clip` Cloud Function with that same token (section 8). The dashboard is a separate client of the same database. No bespoke transport or message broker is involved; the contract at this tier is just the database schema and the function's request format.

11 Security and privacy

Privacy is a design property of Meowtion, not a setting bolted on afterwards. The strongest guarantees come from where work happens (on the collar) and how data is scoped (per owner, by enforced rules).

11.1 Audio never leaves the collar

The microphone is used only for on-device classification. Audio is processed on the collar and discarded; it is never written to flash in normal operation and never transmitted in production mode. The collar's uplink is a classification result (a class index and a confidence), not a sensor stream. The one exception is deliberate and owner-driven: in training mode the owner chooses to capture clips to teach a behaviour, and those clips are stored privately to their own account and are never world-readable, even for the public demo.

11.2 Per-owner isolation

All of an owner's data lives under `users/<uid>/`, and the database and storage rules scope every read and write to the owning account, so owners (and their token-authenticated devices) touch only their own data. Isolation is enforced by the backend, not the client. The Storage rules go further and deny *all* direct client writes: training clips are written only by the `upload_clip` function and are readable only by their owning account, and model blobs are public-read (so the token-only station can fetch one) but writable only by the trainer.

11.3 Devices hold tokens, not credentials

A station authenticates with a scoped, revocable per-device token rather than the owner's password, and a collar holds nothing at all. A lost or compromised device therefore cannot expose the owner's credentials, and it can be revoked simply by deleting its token from the account, with no password change and no effect on the owner's other devices.

11.4 The demo is read-only by enforcement

The public demo reads a single designated demo account's data, which the rules make world-readable, but that account is write-locked at the backend: the demo cannot change any data or trigger any function. The read-only behaviour is enforced by the rules, not merely hidden in the interface, so it holds even against a hand-crafted client.

11.5 What is and is not a secret

The Firebase web API key shipped in the static front end is a public client identifier, not a secret; security is enforced by Authentication and the rules, not by hiding the key. No service-account key is present in the repository, and the functions run on their ambient service account. Device Wi-Fi credentials and firmware secrets are excluded by `.gitignore`. Everything required to understand and build the system is tracked; only true secrets are not.

11.6 Owner data rights

The account page lets an owner export everything stored about their cats and delete their account, devices, and data. This supports the access and erasure expectations of data-protection regimes such as the UK GDPR. The bundled privacy policy is a starting point and is marked for legal review before any public launch.

12 Future work

The system runs end to end on real hardware: record, label, train, deliver over the air, classify on-device, and display live. The following items are known and planned, roughly in priority order.

Idle/other class

Retrain with an explicit “neither” class so an idle cat is not forced into one of the trained behaviours. This is a data and modelling task, not a firmware change (section 7).

Activate the audio confirm stage

The cascade is built for IMU-then-audio confirmation; finish wiring the audio second stage into production so ambiguous motion is resolved by sound on-device (section 6).

Real classifier in telemetry

Switch the advertised telemetry from the simulated behaviour state machine to live classifier output once a trained model is loaded on the collar (section 10).

Battery characterisation

Measure real collar runtime across a representative behaviour mix and tune the duty cycle of the audio path against it.

More behaviours

Extend the captured and labelled set beyond eat/drink/purr to scratching, litter-tray use, grooming, and play, exercising the label-agnostic pipeline at scale.

Multi-cat disambiguation

Distinguish multiple cats in one household, currently framed as a single-cat, within-subject problem.

Firmware over-the-air

Today only models are delivered over the air; extend the mechanism to full firmware updates.

Publication

Write up the confidence-gated cascade as a short preprint and a venue paper.

13 References

- [1] Zephyr Project. *Zephyr RTOS Documentation*. <https://docs.zephyrproject.org>. Accessed 2026.
- [2] Espressif Systems. *ESP-IDF Programming Guide*. <https://docs.espressif.com/projects/esp-idf>. Accessed 2026.
- [3] Robert David et al. ‘TensorFlow Lite Micro: Embedded Machine Learning for TinyML Systems’. In: *Proceedings of Machine Learning and Systems (MLSys)* 3 (2021). arXiv: [2010.08678](https://arxiv.org/abs/2010.08678).
- [4] Liangzhen Lai, Naveen Suda and Vikas Chandra. ‘CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs’. In: *arXiv preprint* (2018). arXiv: [1801.06601](https://arxiv.org/abs/1801.06601).
- [5] Pete Warden and Daniel Situnayake. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O’Reilly Media, 2019. ISBN: 9781492052043.
- [6] Google. *Firebase Documentation*. <https://firebase.google.com/docs>. Accessed 2026.
- [7] Snowflake. *Streamlit Documentation*. <https://docs.streamlit.io>. Accessed 2026.